CSE 719 — Distributed & Cloud Computing

Lecture 5: Replication Control, 2PC & Multicast Ordering

Question Analysis & Precise Solutions (2022–2024)

Dr. Atiqur Rahman · University of Chittagong

Exam: 1 July 2026

Contents

Reference: Core Concepts

Replication

Definition: An object has identical copies (replicas), each maintained by a separate server.

Why replication?

- **Fault-tolerance:** k replicas $\Rightarrow$ tolerate failure of any $(k-1)$ servers.
- **Load balancing:** read/write load spread across k replicas $\Rightarrow$ load reduced by factor k.
- **Availability:** $1-f^k$ vs $(1-f)$ with no replication (where f = per-server failure probability).

Availability table:

f	No replication	$k=3$	$k=5$
0.1	90%	99.9%	99.999%
0.05	95%	99.9875%	7 Nines
0.01	99%	99.9999%	10 Nines

Two challenges:

1. **Replication transparency:** clients should not know multiple replicas exist. Achieved via *front-end* servers that route requests.
2. **Replication consistency (one-copy serializability):** concurrent transactions on replicated objects must behave as if executed serially on a single logical copy.

Passive vs Active Replication

	Passive (Primary-Backup)	**Active**
Update path	All writes $\rightarrow$ primary $\rightarrow$ backups	All FEs multicast to all replicas
Ordering	Primary provides total order	Requires ordered multicast
Failure	Run election if primary fails	Continue — no single point
Correctness	Primary enforces serial order	Total-order multicast + replicated state machine

Active replication requires each FE to multicast to the entire replica group. Ordering choices: FIFO, Causal, Total, or Hybrid. For strong consistency: Total-order (or Hybrid-Total) + Replicated State Machine. For fault handling: Virtual synchrony (reliable ordered multicast over changing membership).

Multicast Ordering Types (for 2024 Q4c)

Ordering	Guarantee	Use case
FIFO	Messages from *same sender* delivered in send order at all receivers	Best-effort; sender-local order only
Causal	If M_1 causally precedes M_2, all receivers deliver M_1 before M_2	Collaborative editing, social feeds
Total (Atomic)	All receivers deliver *all* messages in the same order	Replicated state machines; requires coordinator or token
Hybrid (*-Total)	Total ordering combined with FIFO or Causal (e.g., FIFO-Total)	Active replication with causality

Why ordering matters for overlapping groups: Two groups $G_1 = \{A, B\}$ and $G_2 = \{B, C\}$ both receive messages. Without total ordering, node B (in both groups) may see messages in a different order from A or C, causing inconsistency. Total ordering ensures all nodes — including those in multiple overlapping groups — agree on delivery order.

Two-Phase Commit (2PC)

Problem: A transaction T touches objects on servers $S_1 \dots S_n$. Must ensure atomicity: **all** commit or **none** commit (= Atomic Commit problem = Consensus).

Phase 1 — Prepare:

1. Coordinator sends `PREPARE` to all participating servers.
2. Each server writes tentative updates to **stable storage** (disk) — right before voting.
3. Each server responds `YES` (can commit) or `NO` (cannot commit).

Phase 2 — Commit or Abort:

1. If coordinator receives `YES` from *all* servers within timeout: send `COMMIT`; each server applies updates from disk to store; replies `OK`.
2. If any `NO` vote *or* timeout before all votes: send `ABORT`; each server discards tentative updates.

Key invariants:

- A server that voted `YES` *cannot* commit or abort unilaterally — it must wait for the coordinator's decision.
- A server that voted `NO` can abort immediately (it knows the transaction will abort).
- Saves before voting ⇒ crash-safe: updates survive a server crash and can be applied (or rolled back) after coordinator sends Phase 2 decision.

Failure handling in 2PC:

Failure type	Response
Server crash (before voting)	Server aborts on restart (no vote committed)
Server crash (after YES vote)	On restart: wait for coordinator or query its decision
Prepare message lost	Server timeouts ⇒ votes NO ⇒ coordinator aborts
YES/NO message lost	Coordinator timeouts ⇒ aborts (pessimistic)
Commit/Abort message lost	Server polls coordinator repeatedly until answer received
Coordinator crash	Logs all decisions on disk; new coordinator (re-elected) takes over

2PC vs 1PC

One-phase commit shortcomings:

- Server with the object has no say in whether to commit or abort. If data is corrupted, the server cannot commit — but other servers already did.
- A server may crash before receiving the commit message, leaving its updates in memory (lost).

2PC fixes both: Phase 1 gives every server a vote; data is persisted to disk before voting.

2024 — Q2a (1.5 marks, partial — consensus definition)

Q2(a): Define consensus algorithms. Differentiate between logical concurrency and physical concurrency. *[1.5 marks]*

Consensus Algorithm. A consensus algorithm allows a set of distributed processes to **agree on a single value** (or decision), even when some processes may fail or messages may be lost. All non-faulty processes must eventually decide the same value, and the decided value must have been proposed by at least one process. *Example:* Paxos, Raft, 2PC (for atomic commit).

Logical Concurrency. Multiple tasks (threads, coroutines) are *logically active at the same time* but may be time-multiplexed on a single CPU. The OS scheduler switches between them; no true simultaneous execution is required. *Example:* 100 threads on a single-core CPU.

Physical Concurrency. Multiple tasks execute on *truly separate hardware units* (multi-core, multi-CPU, multi-machine) simultaneously. *Example:* MapReduce running 1000 Map tasks in parallel across 1000 machines.

Key difference: Logical concurrency is about the *appearance* of simultaneous progress (interleaving); physical concurrency is *actual* simultaneous execution. In distributed systems, the combination of both (logical scheduling + physical parallelism) is what makes consistency control non-trivial: even logically sequential operations can arrive at servers out of order.

Exam pattern: 2024 Q2a (1.5 marks). Very short answer expected. Name consensus, give one-line definition, one-line example. Then 2 sentences on logical vs physical. Do not over-explain.

2022 — Q3a, Q3b, Q3c (3+3+3 = 9 marks)

Q3: Consider a scenario where a system handles ACID transactions. Discuss the shortcoming of each of the following approaches when a crash occurs:
(a) Writing updates directly to disk for every transaction. *[3 marks]*
(b) Buffering updates in memory and flushing to disk every 50 transactions. *[3 marks]*
(c) Keeping data in memory and maintaining a separate log file on disk. *[3 marks]*

(a) Direct-to-Disk Write Per Transaction.

Each transaction writes its updates immediately and directly to disk before committing.

Shortcomings:

1. **Extremely slow performance.** Disk I/O is $\sim 100{,}000\times$ slower than memory access. Every write operation blocks until the disk controller confirms the write. For high-throughput systems this makes each transaction take milliseconds instead of microseconds.
2. **No atomicity on partial writes.** A transaction that touches multiple objects writes them one at a time. If a crash occurs after some objects are written but before all are, the database is left in a *partial state*: some updates are on disk and others are lost. There is no mechanism to undo the already-written objects.
3. **No rollback capability.** There is no journal or undo log. If the transaction must be aborted (e.g., due to a conflict), the already-committed disk writes cannot be rolled back.

(b) Memory Buffer — Flush to Disk Every 50 Transactions.

Updates are held in main memory and written to disk only after every 50 transactions complete.

Shortcomings:

1. **Durability violation (violates D in ACID).** If the system crashes before the batch flush, all up to 50 buffered transactions are lost entirely, even if they had already returned "committed" to the client. Clients believe their data is safe; it is not.
2. **Large inconsistency window.** The gap between the last flush and the current time represents a window of lost work. The larger the batch size, the more data is at risk.
3. **State after crash is inconsistent.** On restart, the system knows the state as of the last flush point, but that state may not reflect all committed transactions — creating a mismatch between what clients were told (committed) and what actually persists.
4. **Unfair: slow transactions penalise fast ones.** Transactions 1–49 cannot be considered durable until transaction 50 triggers the flush — their durability window depends on the timing of the 50th transaction.

(c) Memory Data + Disk Log File (Write-Ahead Log).

All active data lives in memory; every update is first appended to a durable log file on disk before being applied to in-memory state (write-ahead logging, WAL).

Shortcomings:

1. **Long recovery time after crash.** On restart, the system must *replay* all log entries from the last checkpoint to reconstruct in-memory state. For large or busy systems, this log replay can take minutes or longer, causing extended downtime.
2. **Log file grows unboundedly.** Without periodic compaction (checkpointing), the log grows without limit. Checkpointing itself is expensive: it requires flushing in-memory state to disk and may block operations.
3. **Double-write overhead.** Every write incurs two disk operations: one to the log and one to eventually persist the data itself. This doubles disk I/O under write-heavy workloads.
4. **Single log file is a bottleneck / single point of failure.** If the log disk fails, all in-memory state since the last checkpoint is unrecoverable. The log must reside on reliable (possibly replicated) storage, increasing complexity and cost.

Why WAL is still the best of the three: Despite its shortcomings, WAL (approach c) is superior to (a) and (b) because it provides *durability* (log is on disk) and *atomicity* (can replay or undo from log) while keeping active data operations fast (in-memory). The shortcomings of (a) and (b) are more fundamental.

Exam pattern: 2022 Q3a–c (3 marks each). Each part expects ∼2–3 distinct shortcomings with brief explanations. The examiner is testing understanding of the ACID Durability property and why naive persistence approaches fail. Use terms: **ACID, Durability, atomicity, rollback, write-ahead log, checkpoint, inconsistency window**.

2024 — Q2c (3 marks)

Q2(c): What is the atomic commit protocol? Explain the two-phase commit protocol for nested transactions. *[3 marks]*

Atomic Commit Protocol. An atomic commit protocol ensures that a distributed transaction that spans multiple servers either **commits at all servers** or **aborts at all servers**. No partial commit is allowed (= the Atomicity property of ACID in a distributed setting). This is an instance of the **consensus problem**: all participating servers must agree on one decision (commit or abort) despite possible message losses and crashes.

Two-Phase Commit (2PC).

Phase 1 — Prepare/Voting:

1. The **coordinator** sends a `PREPARE` message to all participating servers.
2. Each server:
   (a) Writes its tentative updates to **stable storage** (disk) — before voting.
   (b) Sends `YES` (ready to commit) or `NO` (cannot commit, e.g., data corrupted or constraint violated) back to the coordinator.

Phase 2 — Decision:

1. **All YES received within timeout:** coordinator sends `COMMIT`; all servers apply updates from disk to permanent store and reply `OK`.
2. **Any NO or timeout:** coordinator sends `ABORT`; all servers discard their tentative updates.

2PC for Nested Transactions.

A nested (or sub-) transaction is a transaction launched within the scope of a parent transaction. The key property: a sub-transaction may commit locally, but its commit is *tentative* — if the parent transaction aborts, all sub-transactions are also rolled back.

Protocol:

1. **Phase 1 — Prepare (top-down):** The top-level coordinator sends `PREPARE` to each participating server. Each server that has running sub-transactions must first *complete all sub-transactions locally*: committed sub-transactions are held as tentative; any failed sub-transaction triggers a local `NO` vote. Each server then votes `YES` (all sub-transactions succeeded and data is on stable storage) or `NO` (at least one sub-transaction failed).
2. **Phase 2 — Commit or Abort (top-down):**
   - If all servers vote `YES`: coordinator broadcasts `COMMIT`; all servers make their tentative sub-transaction results permanent.
   - If any server votes `NO`: coordinator broadcasts `ABORT`; all servers roll back *all* sub-transactions, including those that had committed locally.

Key rule: A locally-committed sub-transaction is *not* final until the parent transaction commits. The outer 2PC provides the global atomicity guarantee across all participating servers and their nested sub-transactions.

Exam pattern: 2024 Q2c (3 marks). Breakdown: ∼0.5 marks for atomic commit definition, ∼1 mark for basic 2PC (Phase 1 + Phase 2), ∼1.5 marks for the nested-transaction extension. Key point the examiner wants: locally-committed sub-transactions can still be rolled back if the parent aborts — that's what makes 2PC for nested transactions distinct.

2024 — Q4c (3 marks)

Q4(c): Briefly explain the different types of ordering of multicast messages in overlapping groups. *[3 marks]*

In a distributed system with **overlapping process groups** (a process may belong to more than one group simultaneously), multicast messages must be delivered in a consistent order to prevent replicas in multiple groups from diverging.

1. FIFO Ordering. Messages sent by the *same sender* are delivered at all receivers in the order they were sent.

- If process P sends M_1 then M_2, all group members deliver M_1 before M_2.
- Messages from *different senders* may arrive in any order.
- Problem for overlapping groups: process B in both G_1 and G_2 sees messages from each group in sender order, but cannot guarantee global ordering across different senders.

2. Causal Ordering. If message M_1 *causally precedes* M_2 (i.e., M_2 was sent after M_1 was received, or they share a causal chain), then all receivers deliver M_1 before M_2.

- Stronger than FIFO: captures happen-before relationships across senders.
- Implemented using vector clocks.
- Still allows concurrent messages (not causally related) to be delivered in different orders at different nodes.

3. Total Ordering (Atomic Broadcast). All processes in the group deliver *all* messages in **exactly the same order**, regardless of sender or causal relationship.

- Achieved via a sequencer (central token), consensus (Paxos/Raft), or a group ordering protocol.
- Critical for overlapping groups: a process B in two groups G_1 and G_2 will apply all updates in the same sequence as every other member of both groups.
- Required for active replication with Replicated State Machines — all replicas receive the same sequence of updates $\Rightarrow$ arrive at the same state.

4. Hybrid Ordering (e.g., FIFO-Total or Causal-Total). Combines total ordering with FIFO or causal ordering. Ensures both sender-order preservation *and* global agreement on delivery sequence. Used in active replication systems that also need causal consistency.

Why ordering matters for overlapping groups: Consider $G_1 = \{A, B\}$ and $G_2 = \{B, C\}$. Node B belongs to both. Without total ordering, B may deliver a write to object X from G_1 after C has already processed a read from G_2 that should have seen the update, causing inconsistency. Total ordering ensures all nodes — including those in multiple overlapping groups — agree on the same global delivery sequence.

Exam pattern: 2024 Q4c (3 marks). The examiner wants: (1) FIFO — same sender, send order; (2) Causal — happen-before preserved; (3) Total — same order at all receivers; and the overlapping-groups motivation. 3 marks $\rightarrow$ roughly 1 mark per ordering type. Keep definitions one sentence each, then explain the relevance to overlapping groups.

Summary Table

Year	Q	Marks	Topic	Key answer
2022	Q3a	3	ACID crash: direct-to-disk	Slow I/O + partial write + no rollback
2022	Q3b	3	ACID crash: batch-50-memory	Durability violated; crash loses 50 txns
2022	Q3c	3	ACID crash: memory + log	Log grows; slow recovery; double-write
2024	Q2a	1.5	Consensus + logical/physical	Agree on one value; logical=interleaved, p
2024	Q2c	3	Atomic commit + 2PC nested	Phase 1: prepare+vote; Phase 2: commit/
2024	Q4c	3	Multicast ordering types	FIFO / Causal / Total / Hybrid; overlapp
Total covered this PDF		**16.5 marks**		

Pending (answered in L9 / L10 solutions):

- **Consistency models** (weak/causal/release/strong, quorums, linearizability): 2020 Q4a/b, 2021 Q2a–c, 2022 Q4d, 2024 Q2b ⇒ Covered in DSM (Lecture-09) wiki page and solutions.
- **ACID definition + deadlock + locking + WFG + timestamp ordering**: 2021 Q1d, 2021 Q6a/b, 2022 Q6a, 2022 Q7a/b/c, 2024 Q3c, 2024 Q6a, 2024 Q8a/b ⇒ Covered in RPC & Concurrency (Lecture-10) solutions.

Key terms for this lecture in exam: replication transparency, one-copy serializability, passive/active replication, FIFO/Causal/Total ordering, virtual synchrony, atomic commit, Two-Phase Commit, coordinator, prepare, voting, one-phase-commit shortcomings, write-ahead log.